

Cloud POS SaaS — Full Product Planning Document

Multi-shop, multi-industry POS platform for small & medium retail in Pakistan and beyond

Quick note before we start: **Figma's connector here can only read/export from existing Figma files (assets, video, motion data) — it has no API to create new files or draw diagrams.** So it can't generate this mindmap for you. Instead, the mindmap and flow diagrams below are in Mermaid.js — paste them into mermaid.live or a Figma Mermaid plugin (FigJam has one) and you'll get an editable visual instantly.

Market check (Jul 2026): Pakistani POS competitors (LookPOS, Granet Pro, Websol, Moneypex) price PKR 2,999–9,999/mo, emphasize **offline-first billing, FBR/PRA tax compliance, JazzCash/Easypaisa integration, and Urdu support** — these are baseline expectations, not differentiators. Your edge has to come from cross-industry flexibility (one platform, many shop types) and better UX/reporting.

1. Product Vision

Problem: Small/medium shop owners in Pakistan (and similar markets) run on manual registers, Excel, or disconnected desktop billing software. They don't know real-time stock, profit, or dues until it's too late. Desktop POS breaks when the PC dies, can't be checked from home, and doesn't scale to a second branch.

Who it helps: Grocery, pharmacy, cosmetics, mobile accessories, clothing, hardware, electronics, stationery, and general shops — owners who want visibility and control without hiring an accountant or IT person.

Why owners need it: One login shows sales, stock, dues, and profit from anywhere. No more manual khata, no more guessing what's low in stock, no more year-end surprises about missing inventory.

Path to SaaS: Multi-tenant cloud app, subscription-based, shop owner self-registers, picks a plan, invites staff, starts billing same day. Recurring revenue, no per-shop custom installs.

Differentiator vs basic billing software: Basic tools just print receipts. This is an operating system for the shop — inventory ledger, dues tracking, returns, cash reconciliation, staff accountability (audit logs), and reports that answer "am I making money" — while staying industry-agnostic through configurable units, categories, and stock rules (batch/expiry for pharmacy, variants for clothing, IMEI/serial for mobiles).

2. Target Users

User	Main Problems	Daily Tasks	Features Needed	Reports They Care About
Shop Owner	No visibility into profit, stock, staff honesty	Check dashboard, approve big returns/expenses, review cash closing	Dashboard, all reports, staff roles, audit log	Profit, daily sales, dues, expense
Manager	Coordinating staff, stock, and daily targets	Approve returns, manage purchases, handle escalations	Purchase mgmt, return approval, staff performance	Staff performance, stock, sales
Cashier	Speed at checkout, handling cash accurately	Billing, cash register open/close, basic returns	Fast POS screen, barcode scan, shift closing	Own shift sales, cash register report
Inventory Staff	Manual stock counts, missed low-stock, expiry	Receive purchases, adjust stock, check alerts	Stock ledger, adjustments, low-stock/expiry alerts	Stock report, expiry report
Accountant	Reconciling dues, expenses, taxes at month-end	Review payments, expenses, supplier/customer dues	Dues, expense tracking, tax report	Supplier due, customer due, tax, expense
Admin / SaaS Owner	Managing tenants, billing, platform health	Monitor subscriptions, support tickets, usage	Tenant management, subscription plans, system logs	Revenue (MRR), active shops, churn

3. Target Shop Types

Shop Type	Required Features	Special Stock Needs	Return Handling	Reporting Needs	MVP Priority
Grocery	Barcode, weight-based units, quick billing	Expiry dates, bulk units (kg/pack)	Rare, mostly damaged goods	Fast-moving item report	High

Shop Type	Required Features	Special Stock Needs	Return Handling	Reporting Needs	MVP Priority
Pharmacy	Batch/expiry tracking, prescription note	Batch number + expiry mandatory	Damaged/expired returns to supplier	Expiry report critical	High
Cosmetics	Barcode, brand-wise catalog	Expiry, batch (less strict than pharmacy)	Damaged/unused returns	Brand-wise sales	Medium
Mobile Accessories	Barcode, variant (color/model)	IMEI/serial optional for phones	Warranty-based returns	Product-wise sales	Medium
Clothing	Size/color variants, seasonal pricing	Variant-level stock (size × color)	Exchange common, size swap	Category/season sales	High
Hardware	Loose/bulk units (per piece, per foot)	Mixed unit types	Rare, mostly defect-based	Stock report	Medium
Electronics	Serial/IMEI tracking, warranty	Serial number mandatory	Warranty claims, defect returns	Warranty/return report	Medium
Stationery	Simple SKU, school-season demand	Basic stock only	Rare	Category sales	Low
General Store	Mixed of all above, simplest config	Flexible units	Basic returns	Daily sales	High

Build MVP around Grocery + Pharmacy + Clothing + General — they force you to solve expiry, variants, and units early, which then unlocks all other shop types.

4. Core Modules

Module	Purpose	CRUD	Key Validations	Business Rules	Edge Cases
Authentication	Secure login per user/tenant	Create user, reset password	Unique email per tenant, strong password	Session tied to shop context	User belongs to multiple shops
Shop Setup	Define business profile	Create/edit shop	Shop name, currency, tax ID unique	One owner per shop, transferable	Shop deleted with active subscription
Roles & Permissions	Control access	CRUD roles, assign permissions	Can't remove last admin	Role scoped per shop	Custom role creation
Product Management	Catalog of sellable items	Full CRUD	SKU/barcode unique per shop	Variant support, min-stock threshold	Duplicate barcode across shops (allowed)
Category Management	Organize products	Full CRUD	Unique name per shop	Nested categories optional	Category deleted with products linked
Brand Management	Track manufacturer/brand	Full CRUD	Unique per shop	Optional field on product	—
Unit Management	kg, pcs, box, litre, etc.	Full CRUD	Unit conversion ratio required for multi-unit	Base unit + alt units	Wrong conversion breaks stock math
Supplier Management	Vendor records + dues	Full CRUD	Unique contact	Linked to purchases	Supplier with pending dues can't be deleted
Customer Management	Buyer records + dues/credit	Full CRUD	Phone/unique ID optional	Credit limit per customer	Walk-in customer

Module	Purpose	CRUD	Key Validations	Business Rules	Edge Cases
					(no record)
Purchase Management	Record stock-in from suppliers	Full CRUD	Quantity > 0, valid supplier	Auto stock increase on save	Partial delivery, price change mid-purchase
Sales / POS Billing	Core checkout screen	Create sale, void (with permission)	Stock availability check	Auto stock decrease, ledger entry	Selling below available stock, split payment
Sale Return Handling	Process customer returns	Create return, approve	Valid original invoice (if required)	Restocks (unless damaged), affects cash/credit	Return without invoice, partial item return
Purchase Return Handling	Return to supplier	Create return	Valid purchase reference	Stock decrease, supplier due adjustment	Return after payment already made
Inventory / Stock Ledger	Single source of truth for stock	Read (system-generated)	Every stock change must have a source record	Never allow direct stock edit without ledger entry	Race condition: two cashiers sell last unit
Stock Adjustment	Manual correction (loss/damage/count)	Create	Reason mandatory	Requires manager approval above threshold	Repeated adjustments = fraud flag
Expenses	Track shop running costs	Full CRUD	Category + amount required	Deducted in profit report	Recurring expense templates
Cash Register	Shift open/close reconciliation	Open, close, view history	Opening float required	Close must match expected vs counted cash	Shift left open overnight

Module	Purpose	CRUD	Key Validations	Business Rules	Edge Cases
Reports	Business intelligence	Read only	Date range required	Cached for performance on large data	Report timeout on huge date ranges
Dashboard	At-a-glance summary	Read only	Role-based data scope	Real-time or near-real-time refresh	Slow load with many shops
Audit Logs	Who did what, when	Read only, auto-write	Immutable, no delete	Every sensitive action logged	Log storage growth over time
Settings	Tenant-level configuration	Update	Only owner/admin can edit	Currency/tax locked after first sale	Changing tax rate mid-month
Subscription Plans	SaaS billing	Full CRUD (admin only)	Valid plan limits	Auto-downgrade on non-payment	Shop exceeds plan's branch/user limit
Multi-Branch System	Manage multiple locations under one shop	Full CRUD	Branch-level stock separation	Central owner view across branches	Stock transfer between branches

5. Inventory Logic

Core rule: **stock never changes directly — every change is a ledger entry with a source document.** This is what prevents "why is my stock wrong" support tickets.

Action	Stock Effect	Example	Required Record
Purchase received	+Increase	Buy 100 units from supplier	<code>purchase_items</code> + <code>stock_movements</code> (IN)
Sale completed	–Decrease	Sell 5 units to customer	<code>sale_items</code> + <code>stock_movements</code> (OUT)
Sale return	+Increase	Customer returns 2 units	<code>sale_return_items</code> + <code>stock_movements</code> (IN)
Purchase return	–Decrease	Return 10 defective units to supplier	<code>purchase_return_items</code> + <code>stock_movements</code> (OUT)
Damage/loss	–Decrease	3 units expired/broken	<code>stock_adjustments</code> (reason: damage) + <code>stock_movements</code> (OUT)
Manual adjustment	+/–	Physical count found 2 fewer units	<code>stock_adjustments</code> (reason: count) + <code>stock_movements</code>
Branch transfer out	–Decrease (source branch)	Move 20 units to Branch B	<code>stock_transfers</code> + <code>stock_movements</code> (OUT, source)
Branch transfer in	+Increase (destination branch)	Branch B receives 20 units	<code>stock_transfers</code> + <code>stock_movements</code> (IN, destination)
Low-stock trigger	No stock change	Stock hits reorder threshold	System alert, no ledger entry

Every row in `stock_movements` stores: product, branch, quantity delta, resulting balance, source type, source ID, user, timestamp. This table is your audit trail and the only thing reports should read from — never sum raw sales minus raw purchases live.

6. Product Return Handling

Return Type	Stock	Sales Report	Profit Report	Cash Register	Customer Balance	Audit Log
Full sale return	Restocked (if resellable)	Invoice marked returned	Profit reversed	Cash refunded or credited	Adjusted if store credit	Logged

Return Type	Stock	Sales Report	Profit Report	Cash Register	Customer Balance	Audit Log
Partial sale return	Restocked for returned qty only	Line-item adjusted	Partial profit reversed	Partial refund	Adjusted	Logged
Exchange	Old item in, new item out (two ledger entries)	Both transactions recorded	Net profit recalculated	Price difference settled	Adjusted if applicable	Logged
Damaged product return	NOT restocked (goes to loss)	Marked returned	Full loss (cost written off)	Refund still due to customer	Adjusted	Logged with reason
Refund in cash	Reduces till cash	—	—	Cash out entry	N/A	Logged
Refund as store credit	—	—	—	No cash movement	Customer balance increased	Logged
Return without invoice	Manager approval required	Flagged "no reference"	Estimated cost basis used	Cautious refund (often store credit only)	Adjusted	High-visibility log entry
Return with invoice	Auto-validated	Linked directly	Exact cost/profit reversed	Standard refund	Adjusted	Logged
Manager-approved return	Blocks until approved	—	—	—	—	Approval + approver logged
Reason-tracked return	—	Reason column populated	Feeds into "return report" analytics	—	—	Reason mandatory field

Rule of thumb: **damaged and no-invoice returns are the two riskiest paths** — always require manager approval and always log a reason. This is your main defense against return fraud.

7. Reports

Report	Purpose	Filters	Key Columns	Used By	Business Value
Daily Sales	Today's performance	Date, branch, cashier	Invoice, time, amount, payment type	Owner, Cashier	Spot slow/fast days
Monthly Sales	Trend over month	Month, branch	Daily totals, growth %	Owner	Planning, seasonality
Profit Report	Real margin after cost	Date range, category	Revenue, COGS, gross profit	Owner, Accountant	Core profitability check
Product-wise Sales	Best/worst sellers	Date, category	Product, qty sold, revenue	Owner, Manager	Reorder decisions
Category-wise Sales	Category performance	Date range	Category, revenue, % share	Owner	Merchandising decisions
Stock Report	Current inventory state	Branch, category	Product, qty, value	Inventory staff	Stock valuation
Low-Stock Report	Reorder alerts	Threshold	Product, current qty, min qty	Inventory staff	Prevent stockouts
Expiry Report	Batches nearing expiry	Days-to-expiry	Product, batch, expiry date, qty	Pharmacy/Grocery staff	Prevent expired-stock loss
Return Report	Return patterns	Date, reason, staff	Product, reason, qty, value	Owner, Manager	Detect fraud/quality issues
Purchase Report	Buying history	Date, supplier	Supplier, items, cost	Owner, Accountant	Cost control

Report	Purpose	Filters	Key Columns	Used By	Business Value
Supplier Due Report	Outstanding payables	Supplier, status	Supplier, amount owed, due date	Accountant	Cash flow planning
Customer Due Report	Outstanding receivables	Customer, status	Customer, amount owed, aging	Accountant	Collections
Expense Report	Running costs	Date, category	Category, amount, note	Owner, Accountant	Cost visibility
Cash Register Report	Shift reconciliation	Date, cashier	Opening, closing, expected vs actual	Owner, Manager	Catch cash shortages
Staff Performance	Sales per staff	Date, staff	Staff, sales, transactions, avg ticket	Owner, Manager	Incentives, accountability
Tax Report	Tax owed/collected	Date range	Taxable sales, tax amount	Accountant	FBR/tax filing
Audit Report	Full activity trail	Date, user, action type	User, action, entity, timestamp	Owner, Admin	Security, dispute resolution

8. Dashboard Ideas

Shop Owner Dashboard

- Cards: Today's sales, today's profit, cash in register, pending customer dues, pending supplier dues
- Graphs: 7-day sales trend, category-wise sales pie
- Alerts: Low stock count, expiring items count, unclosed shifts
- Tables: Top 5 products today, recent large transactions
- Shortcuts: New sale, add expense, view reports

Cashier Dashboard

- Cards: My shift sales, my shift transaction count, cash in drawer
- Alerts: Shift not opened / needs closing
- Shortcuts: Start sale, open/close register

Inventory Dashboard

- Cards: Total SKUs, low-stock count, expiring-soon count
- Tables: Low-stock list, expiry list, pending purchase orders
- Shortcuts: New purchase, stock adjustment

Admin/SaaS Dashboard

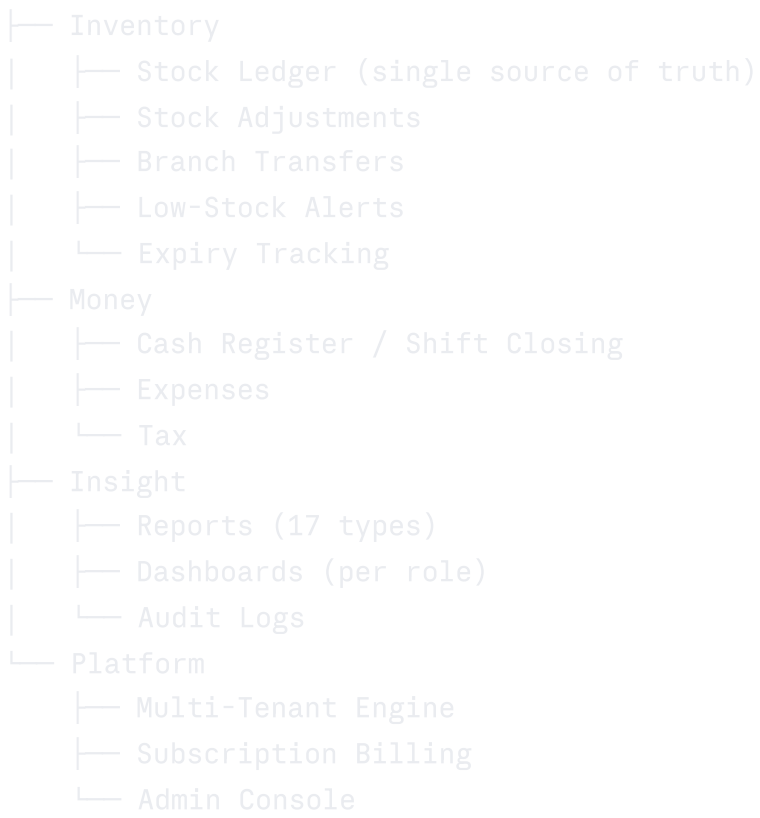
- Cards: Active shops, MRR, new signups this month, churn rate
 - Graphs: Revenue trend, plan distribution
 - Tables: Recently signed-up shops, shops nearing plan limits, support tickets
 - Shortcuts: Manage plans, view tenant list
-

9. Mind Map

```

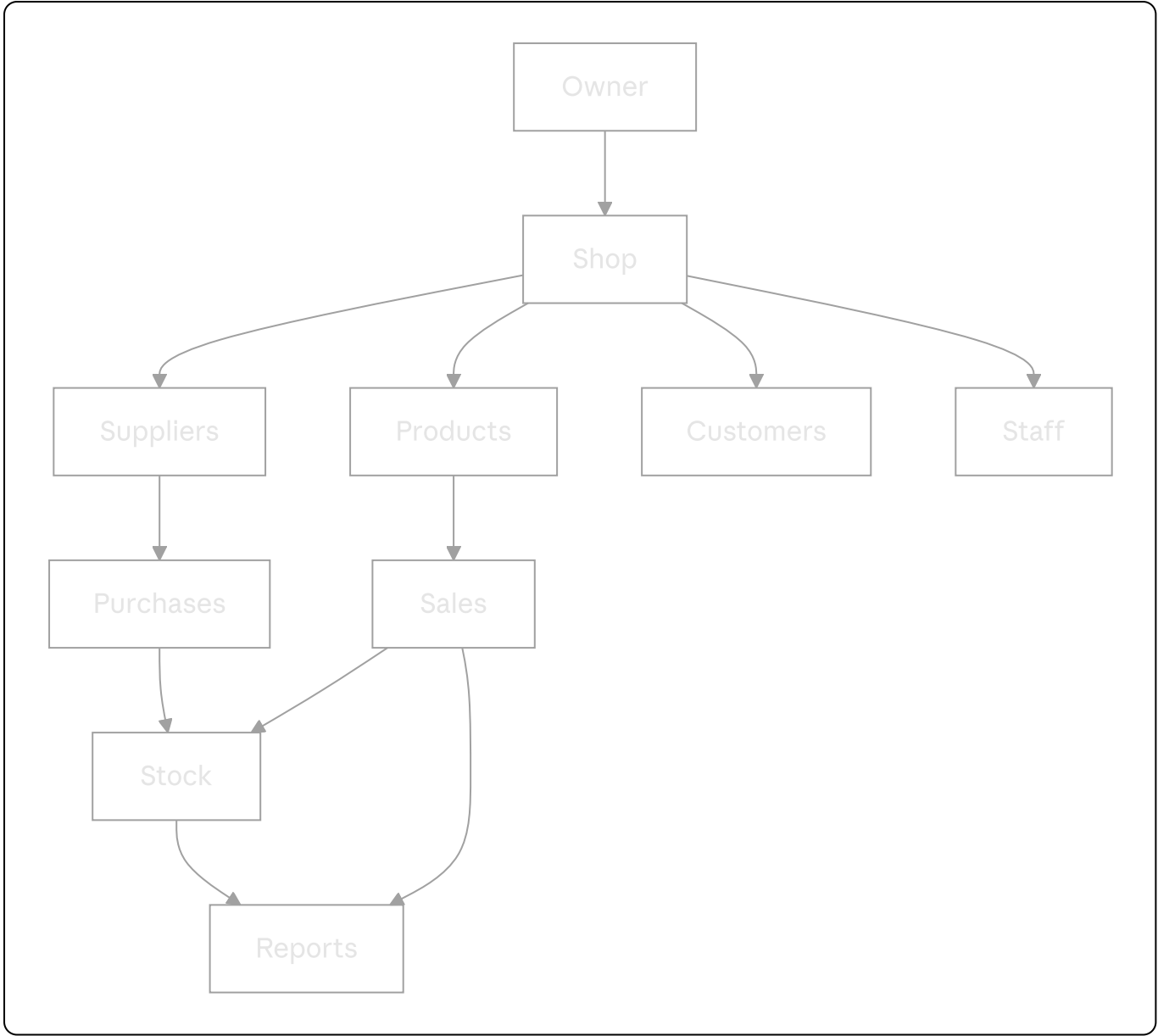
POS Software
├── Core Setup
│   ├── Shop Profile
│   ├── Branches
│   ├── Users
│   ├── Roles & Permissions
│   └── Subscription Plan
├── Catalog
│   ├── Products (CRUD, variants, barcode)
│   ├── Categories
│   ├── Brands
│   └── Units (base + conversion)
├── Parties
│   ├── Suppliers (+ dues)
│   └── Customers (+ dues/credit)
├── Purchasing
│   ├── Purchase Orders
│   ├── Purchase Returns
│   └── Supplier Payments
├── Selling
│   ├── POS Billing Screen
│   ├── Sale Returns / Exchanges
│   ├── Invoice Printing
│   └── Customer Payments

```

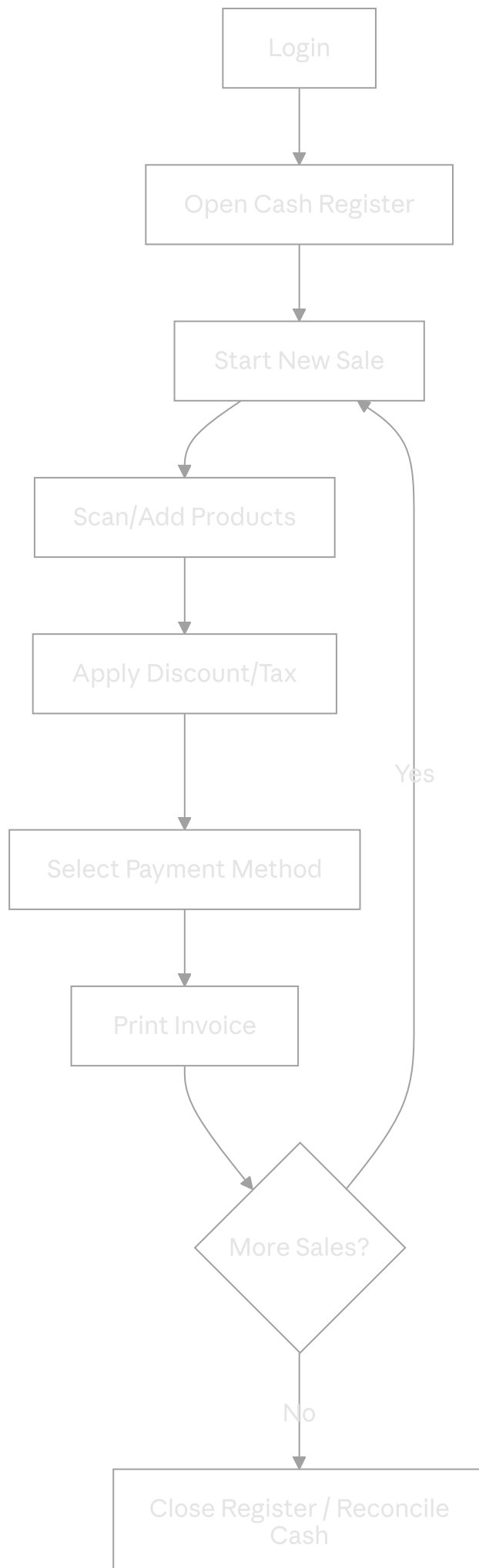


10. Visual Brainstorming Boards (Mermaid)

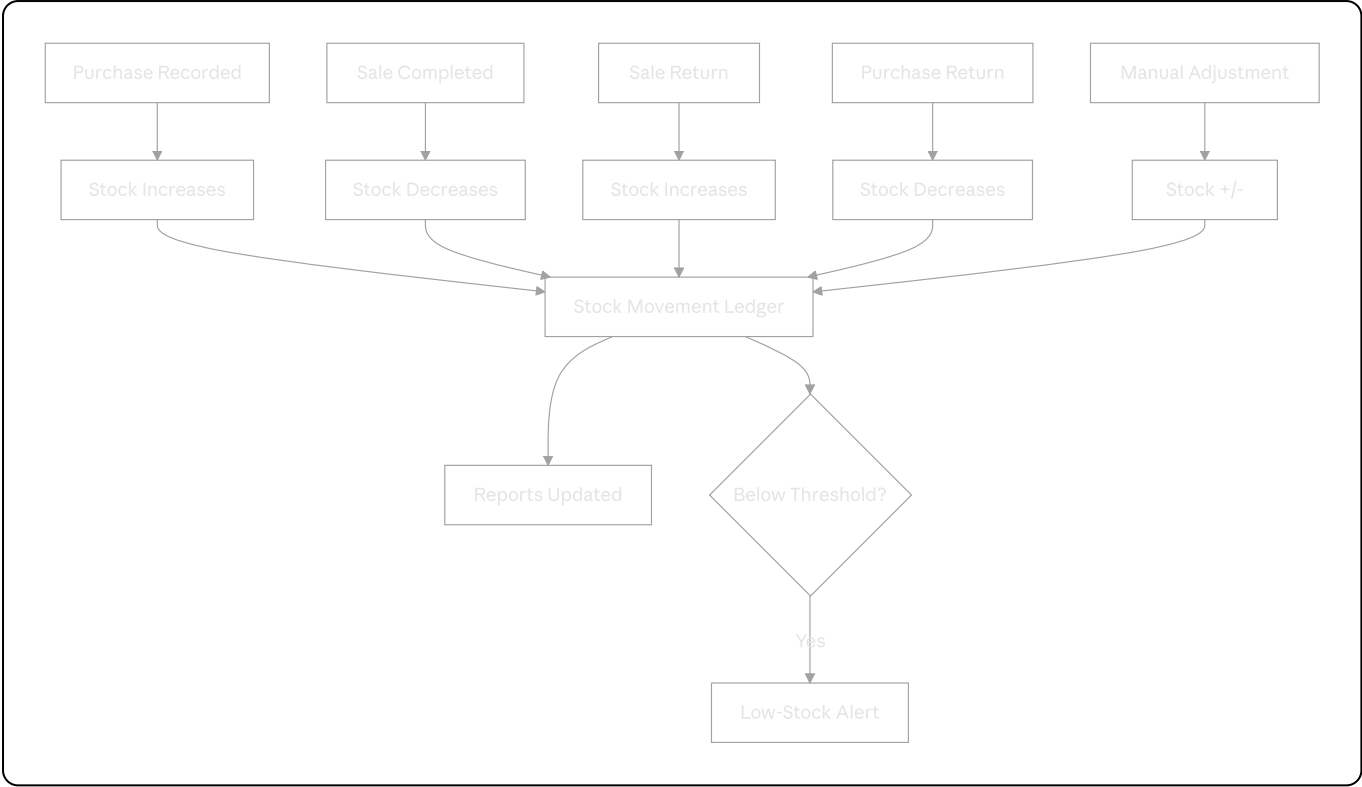
Product Ecosystem



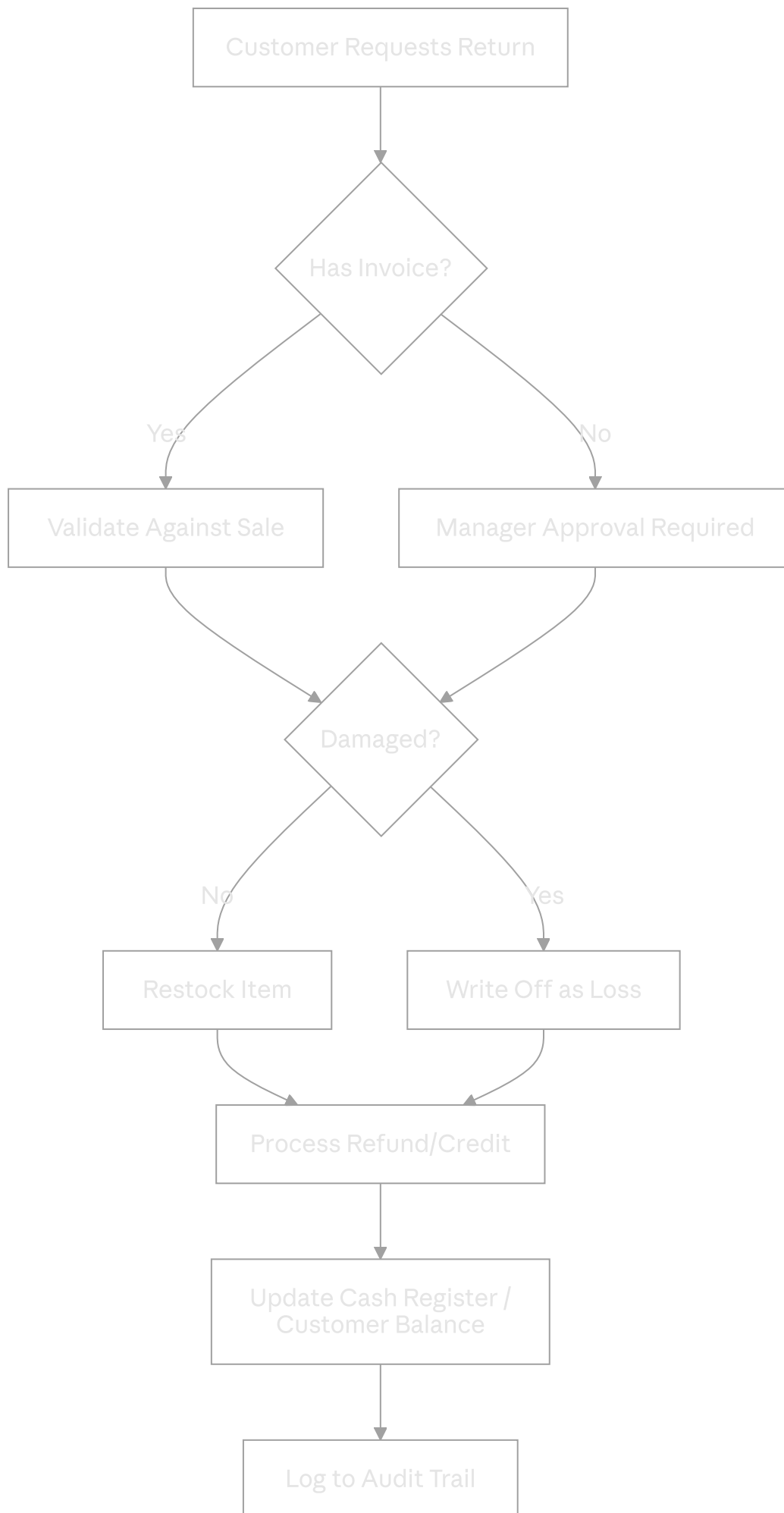
User Flow (Cashier)



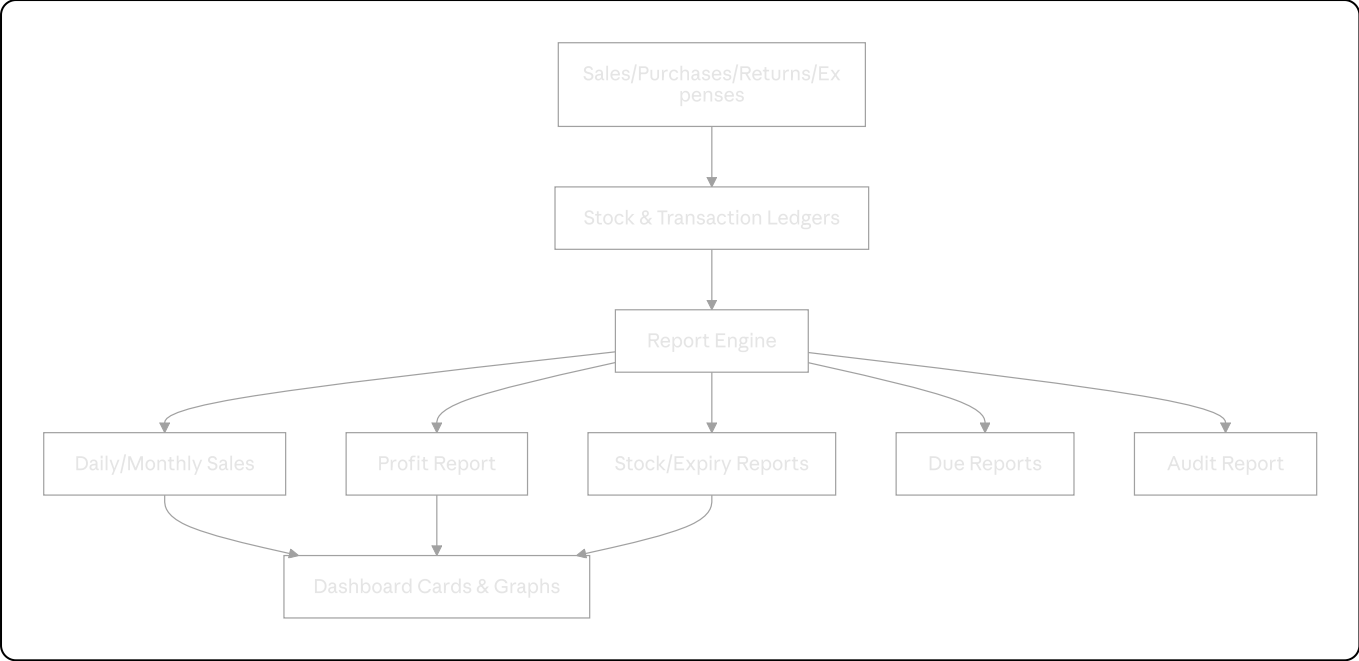
Inventory Flow



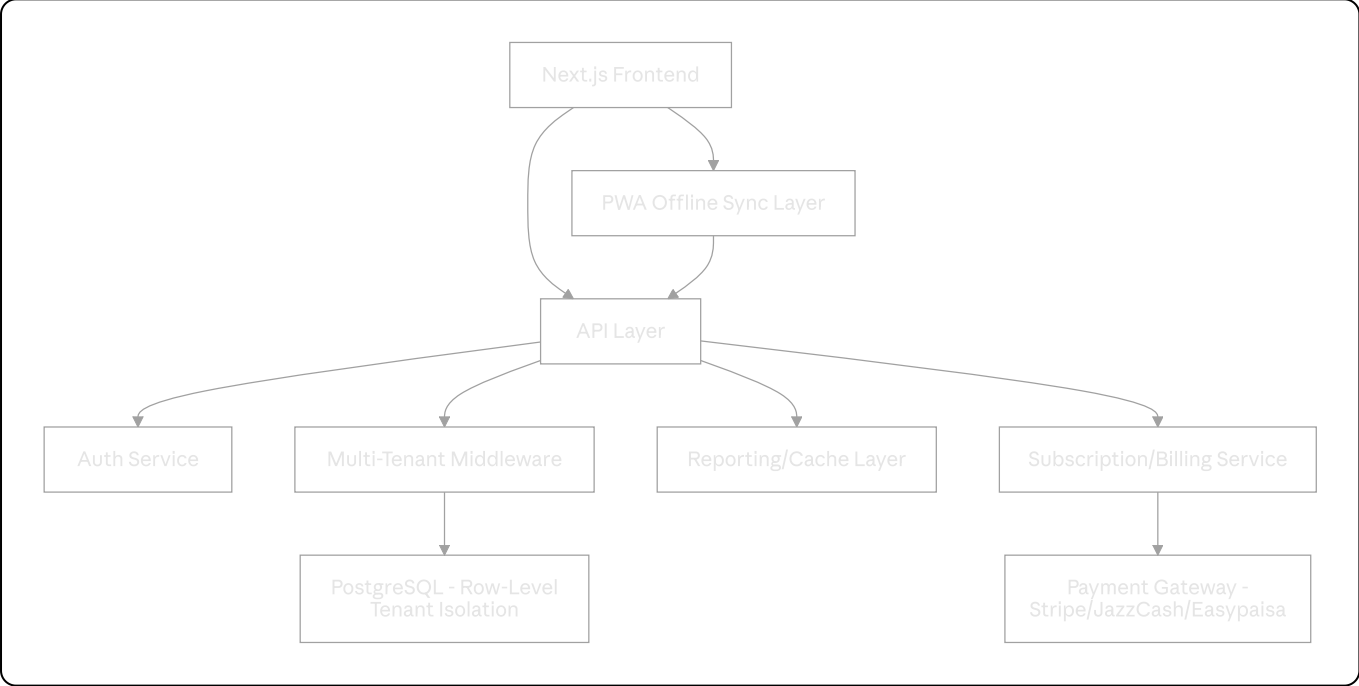
Return Handling Flow



Reporting Flow



SaaS Architecture



11. Feature Prioritization

Feature	Priority	Version	Reason	Complexity
Product/Category/Supplier/Customer CRUD	Must	MVP	Foundation for everything	Low
POS Billing + Invoice Printing	Must	MVP	Core value prop	Medium
Purchase Management	Must	MVP	Needed for stock-in	Medium

Feature	Priority	Version	Reason	Complexity
Stock Ledger + Low-Stock Alerts	Must	MVP	Core differentiator	Medium
Sale Returns	Must	MVP	Daily necessity	Medium
Role-Based Access	Must	MVP	Trust/security	Medium
Cash Register / Shift Closing	Must	MVP	Cash accountability	Medium
Basic Dashboard + Core Reports	Must	MVP	Owner's main reason to buy	Medium
Customer/Supplier Dues	Should	MVP	Common in Pakistani retail (credit sales)	Medium
Expense Tracking	Should	MVP	Needed for profit report	Low
Purchase Returns	Should	V2	Less frequent than sale returns	Medium
Audit Logs	Should	V2	Important but not launch-blocking	Medium
Multi-Branch	Could	V2	Most shops start single-location	High
Batch/Expiry Tracking	Should (pharmacy)	V2	Needed once pharmacy segment targeted	Medium
Variant Support (size/color)	Should (clothing)	V2	Needed once clothing segment targeted	High
Advanced Reports (staff performance, tax)	Could	V2	Value-add, not core	Medium
Offline/PWA Mode	Could	V3	Big complexity, big local-market value	High

Feature	Priority	Version	Reason	Complexity
WhatsApp Alerts	Could	V3	Nice-to-have add-on revenue	Medium
FBR/Tax Integration	Could	V3	Regulatory, high effort, high local value	High
AI Demand Forecasting	Won't (yet)	Future	Needs data volume first	High
Loyalty/Rewards Program	Won't (yet)	Future	Not core to MVP buyer	Medium
Marketplace/Multi-Channel Selling	Won't (yet)	Future	Different product entirely	High

12. MVP Scope

Must-have: Auth + roles, shop setup, product/category/supplier/customer CRUD, purchase entry, POS billing, sale returns, stock ledger + low-stock alerts, cash register, invoice printing, core reports (daily sales, profit, stock, dues), basic dashboard.

Should-have (fast follow, not launch blockers): Expense tracking, customer/supplier dues, purchase returns.

Avoid at start: Multi-branch, batch/expiry system, variant matrix, offline mode, WhatsApp/FBR integrations, AI features — all add real complexity and aren't needed to prove the core loop.

Timeline (realistic, one small team):

- Weeks 1-2: Auth, multi-tenant setup, shop/user/role schema
- Weeks 3-5: Catalog (products, categories, suppliers, customers)
- Weeks 6-8: Purchases + stock ledger
- Weeks 9-11: POS billing + invoice printing
- Week 12: Sale returns
- Week 13: Cash register / shift closing
- Weeks 14-15: Core reports + dashboard
- Week 16: QA, bug fixes, staging
- ~4 months to first sellable version

Testing checklist: Stock math correctness under concurrent sales, role permission boundaries, invoice totals/tax accuracy, return-then-resell flow, cash register reconciliation

edge cases, tenant data isolation (critical — one shop must never see another's data).

Launch checklist: Onboarding flow tested end-to-end, sample data/demo mode, pricing page live, payment gateway connected, support channel ready, backup/restore tested, error monitoring live.

13. Database Brainstorming

Table	Purpose	Key Fields	Relations
users	Login identity	id, name, email, password_hash	→ shop_members
shops	Tenant record	id, name, currency, tax_id, plan_id	→ shop_members, products, subscription
shop_members	User-to-shop link + role	user_id, shop_id, role_id	users, shops, roles
roles	Named permission sets	id, shop_id, name	shop_members, permissions
permissions	Granular access rights	id, role_id, action	roles
products	Sellable items	id, shop_id, name, sku, barcode, unit_id, min_stock	categories, units, brands
categories	Product grouping	id, shop_id, name, parent_id	products
brands	Manufacturer/brand	id, shop_id, name	products
units	Measurement units	id, shop_id, name, base_unit_id, ratio	products
suppliers	Vendors	id, shop_id, name, contact, balance	purchases
customers	Buyers	id, shop_id, name, contact, balance, credit_limit	sales
purchases	Stock-in transactions	id, shop_id, supplier_id, date, total	purchase_items

Table	Purpose	Key Fields	Relations
<code>purchase_items</code>	Line items per purchase	id, purchase_id, product_id, qty, cost	purchases, products
<code>purchase_returns</code>	Stock-out to supplier	id, purchase_id, date, reason	purchase_return_items
<code>purchase_return_items</code>	Line items per return	id, return_id, product_id, qty	purchase_returns, products
<code>sales</code>	Checkout transactions	id, shop_id, customer_id, cashier_id, date, total, payment_type	sale_items
<code>sale_items</code>	Line items per sale	id, sale_id, product_id, qty, price	sales, products
<code>sale_returns</code>	Customer returns	id, sale_id, date, reason, approved_by	sale_return_items
<code>sale_return_items</code>	Line items per return	id, return_id, product_id, qty	sale_returns, products
<code>stock_movements</code>	Immutable stock ledger	id, product_id, branch_id, delta, balance_after, source_type, source_id	products, all transaction tables
<code>stock_adjustments</code>	Manual corrections	id, product_id, qty, reason, approved_by	products, stock_movements
<code>payments</code>	Money in/out against dues	id, party_type, party_id, amount, method, date	customers, suppliers
<code>expenses</code>	Shop running costs	id, shop_id, category, amount, date, note	shops

Table	Purpose	Key Fields	Relations
cash_shifts	Register open/close	id, shop_id, cashier_id, opening_amt, closing_amt, expected_amt	users
audit_logs	Activity trail	id, shop_id, user_id, action, entity, entity_id, timestamp	users, all entities
subscription_plans	SaaS plan definitions	id, name, price, limits (branches/users)	shop_subscriptions
shop_subscriptions	Tenant's active plan	id, shop_id, plan_id, status, renews_at	shops, subscription_plans

14. Business Model

Revenue streams: Monthly/yearly subscription (yearly at ~2 months free, standard SaaS move), per-branch pricing for multi-location, per-user pricing beyond a base seat count, setup/onboarding fee for larger shops needing data migration, premium report add-on, WhatsApp low-stock/due alerts add-on, offline-mode add-on, custom integration/setup fee for chains.

Pricing — Pakistan (matches local market band of PKR 2,999-9,999/mo):

Plan	Price	Includes
Starter	PKR 2,500/mo	1 branch, 2 users, core POS + inventory
Growth	PKR 5,500/mo	1 branch, 5 users, + returns, dues, reports
Pro	PKR 9,500/mo	3 branches, 15 users, + audit logs, multi-branch, WhatsApp alerts
Enterprise	Custom	Unlimited branches/users, dedicated support, custom integrations

Pricing — International (USD, SMB SaaS positioning):

Plan	Price	Includes
Starter	\$19/mo	1 branch, 2 users
Growth	\$39/mo	1 branch, 5 users, reports
Pro	\$79/mo	3 branches, 15 users, full feature set
Enterprise	Custom	Unlimited, dedicated support

Annual billing at ~15-20% discount across all tiers.

15. UI/UX Planning

Sidebar: Dashboard, POS/Billing, Products, Categories, Purchases, Sales, Returns, Customers, Suppliers, Inventory, Expenses, Cash Register, Reports, Staff, Settings.

Dashboard layout: Top KPI cards row → alerts banner (low stock/expiring/unclosed shift) → two-column: sales trend chart + top products table → recent activity feed.

POS Billing screen: Left panel = product search/barcode + category quick-filter grid. Right panel = running cart with qty/price/discount per line, totals, payment method buttons (cash/card/credit), "Complete Sale" primary button. Empty state: "Scan or search a product to start." Error state: stock-insufficient warning inline on the cart line.

Product management screen: Table with columns (image, name, SKU, category, stock, price, status), filters (category, brand, stock status), bulk-action toolbar, "Add Product" button top-right. Empty state: "No products yet — add your first one."

Reports screen: Report picker sidebar/tabs, date-range filter + branch filter at top, export (PDF/Excel) button, chart + table combo view.

Return screen: Search by invoice or customer, item selection with reason dropdown (mandatory), refund method selector, manager-approval banner when no invoice.

Stock adjustment screen: Product search, current stock display, +/- adjustment input, mandatory reason field, approval flag if over threshold.

Settings screen: Tabs for Shop Profile, Branches, Users & Roles, Tax/Currency, Subscription/Billing, Notifications.

16. Technical Architecture

Frontend: Next.js (App Router) + TypeScript + Tailwind + ShadCN UI. Feature-based folder structure (`/features/pos`, `/features/inventory`, etc.), Zustand for local UI state,

TanStack Table for all data grids, React Hook Form + Zod for every form (shared schema between client validation and API).

Backend: Next.js API routes or a separate Node/Express service if you expect heavy load — start with API routes, split out later if needed. Layered: controller → service → repository, so business logic (stock math, return rules) isn't tangled with HTTP handling.

Database: PostgreSQL via Prisma. Every tenant-scoped table carries `shop_id`; enforce isolation with a Prisma middleware or Postgres Row-Level Security — don't rely on the app layer alone for tenant isolation.

Auth: Clerk or Auth.js, with shop membership + role stored in your own `shop_members`/`roles` tables (auth provider handles identity, your DB handles authorization).

Multi-tenant strategy: Shared database, shared schema, `shop_id` scoping on every query (cheapest to run, fine at SMB scale). Revisit schema-per-tenant only if a large enterprise customer demands hard isolation.

Role-based access: Permissions table checked server-side on every mutating endpoint — never trust client-side role checks alone.

Reporting strategy: Reports never do live aggregation over raw `sales`/`stock_movements` at scale — use materialized views or a nightly/hourly rollup table for anything beyond "today," and only query live for real-time dashboard cards.

Deployment: Vercel for frontend/API, Neon or Supabase for Postgres (both support branching for staging). Background jobs (report rollups, alert checks) on a small cron worker (Vercel Cron or a separate lightweight service).

17. Risks and Challenges

Risk	Solution
Wrong stock calculations	Single append-only ledger table (<code>stock_movements</code>) as source of truth; never let two code paths write stock directly
Return abuse	Mandatory reason + manager approval on no-invoice/damaged returns; return-pattern report to flag repeat offenders
Data loss	Automated daily DB backups + point-in-time recovery (Neon/Supabase support this natively)
Slow reports	Pre-aggregated rollup tables, indexed date/shop columns, pagination everywhere

Risk	Solution
Cashier mistakes	Confirmation step before completing sale, easy void within a time window (logged), barcode-first UX to reduce manual entry
Multi-shop data leakage	Row-level <code>(shop_id)</code> scoping enforced at DB level (RLS), not just application code
Internet issues	PWA offline queue for billing (V3), local draft-save so a dropped connection doesn't lose an in-progress sale
Printer issues	Support standard ESC/POS thermal printers via browser print API + PDF fallback
Barcode issues	Support manual SKU entry as fallback whenever scanner fails/unavailable
User training problems	In-app guided onboarding checklist, short video tutorials per module, sandbox/demo mode before going live

18. Final Output

Product summary: A cloud-based, multi-tenant POS platform that gives small and medium shop owners — grocery, pharmacy, cosmetics, clothing, hardware, and general retail — real-time control over sales, inventory, dues, and profit from any device, replacing manual registers and disconnected desktop software.

Positioning statement: One POS platform, every shop type — real inventory, real profit, from anywhere.

Landing page headline: "Run your shop from your phone. Know your stock, your dues, your profit — every single day."

Sales pitch: Stop guessing what's in stock or what customers owe you. This POS tracks every sale, purchase, and return automatically, so your dashboard always shows the real picture — accessible from your shop, your home, or your second branch.

30-second explanation for shop owners: "You open your phone, see today's sales and profit instantly. Every sale updates your stock automatically — no manual counting. When stock runs low, you get an alert before you run out. Customer and supplier dues are tracked so nothing gets forgotten. It works for grocery, pharmacy, clothing, or any shop — one system, one login, from anywhere."

Recommended MVP plan: Build core CRUD + POS billing + stock ledger + cash register + core reports first (~4 months), launch to grocery/general stores, gather real usage data, then layer in pharmacy (batch/expiry) and clothing (variants) as V2 once the core loop is validated.

Final mind map summary: Core Setup → Catalog → Parties (suppliers/customers) → Purchasing → Selling → Inventory Ledger → Money (cash/expenses/tax) → Insight (reports/dashboards/audit) → Platform (multi-tenant + billing). Every module ultimately feeds the stock ledger and the reporting layer — that's the spine of the whole product.